SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CSA04 **COURSE CODE:** Operating Systems

COURSE OUTCOME COVERED

***CO1**: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 1 **Total Marks:100**

Annexure A

Technical Presentation

Code of Conduct:

I Mounnila S P *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mounnila

98/100

Annexure A

Technical Presentation [Any one]

Question 1

Suppose an Operating System is used in a banking system handling thousands of transactions simultaneously.
Prepare a technical presentation explaining:
Types of OS suitable for this scenario
OS structure used
Role of system calls in transaction processing **(BL3)**

Question 2

Consider a multi-user Linux-based server environment supporting file sharing and device access.
Prepare a technical presentation covering:
OS functions involved in resource management
File and device system calls
Interaction between user processes and kernel **(BL3)**

Question 3

Assume a real-time embedded system used in traffic signal control.
Prepare a technical presentation explaining:
Type of OS used and justification
OS structure and components
System calls used for process control and timing **(BL3)**

Question 4

A company uses a distributed computing system across multiple locations.
Prepare a technical presentation discussing:
Type of OS and its characteristics
Architecture of distributed OS
System calls used for communication and coordination **(BL3)**

Question 5

Consider an OS managing process scheduling, memory allocation, and I/O operations in a modern computer.
Prepare a technical presentation including:
Key OS functions and their importance
Interaction between OS components
Examples of system calls for each function **(BL3)**

RUBRICS FOR CALCULATION

Technical Presentation					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness



QUESTION 5 • TECHNICAL PRESENTATION

OPERATING SYSTEM FUNCTIONS AND SYSTEM CALLS

Process scheduling • Memory allocation • I/O operations

Name: Mounnila S P
Register No:192511188

The engineering problem

A modern computer runs many programs at once, has limited RAM, and shares devices such as disks, keyboards, and network adapters. Without coordination, programs could compete for resources, wait unnecessarily, or interfere with one another.

CPU contention

Several ready processes need CPU time. The OS must decide which process runs next.

Memory pressure

RAM is limited. The OS allocates memory, protects address spaces, and may use virtual memory.

I/O coordination

Devices operate at different speeds. The OS manages requests, interrupts, buffers, and completion.

Objective: explain the key OS functions, show how they cooperate, and identify representative system calls used by applications to request OS services.

2. Key OS Functions and Their Importance

Process management & scheduling

Creates and terminates processes; tracks states; schedules CPU time; supports synchronization and inter-process communication. Importance: responsiveness, fairness, and efficient CPU use.

Memory management

Allocates and releases memory; maps virtual to physical addresses; isolates processes; manages paging and swapping. Importance: protection, efficient RAM use, and safe multitasking.

I/O and device management

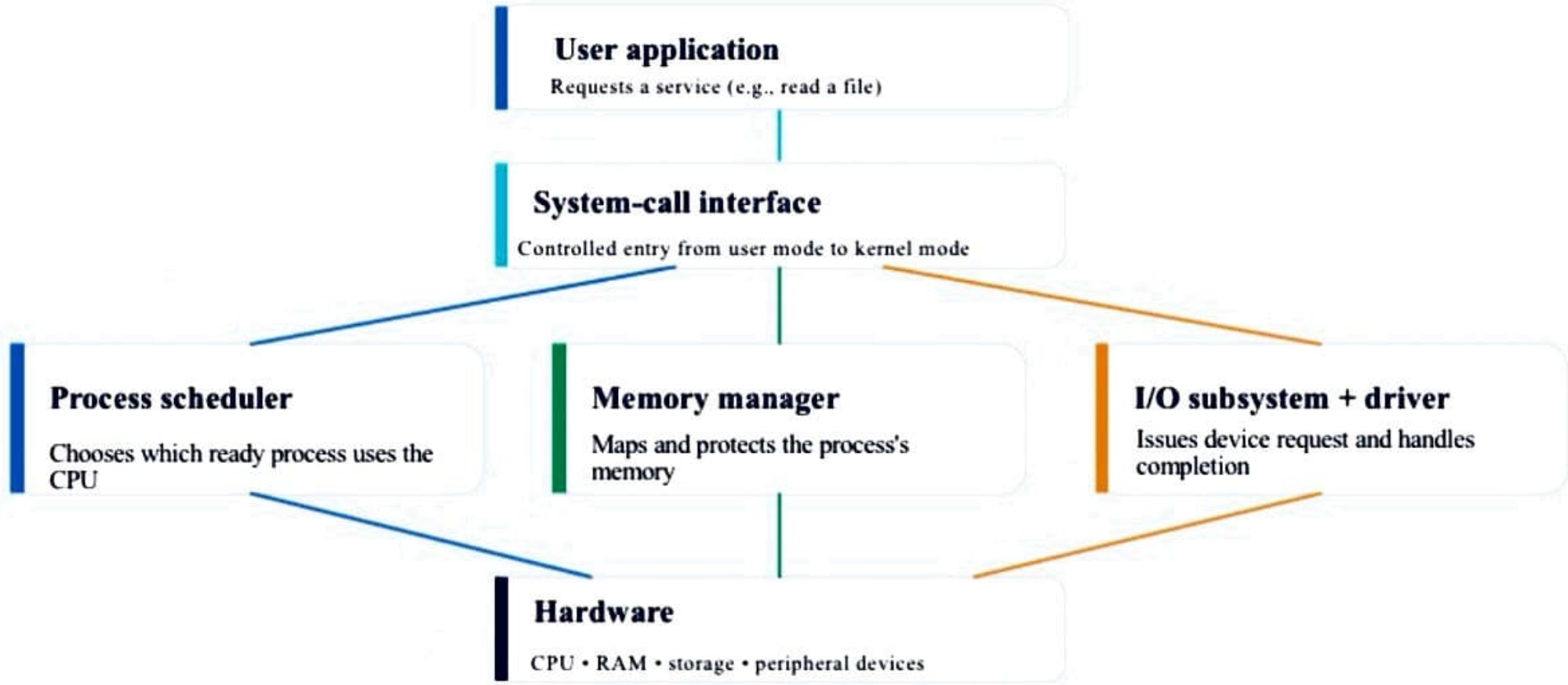
Provides device drivers and a uniform interface; handles interrupts, buffering, caching, and queues. Importance: reliable, efficient access to disks, displays, keyboards, and networks.

File system, security & services

Organizes files/directories; checks permissions; supports networking, error handling, and accounting. Importance: persistent data, controlled access, and system reliability.

These functions are coordinated by the kernel, the privileged core of the operating system.

3. How OS Components Interact



Example: a file read may block the process, let the scheduler run another process, and resume the original process when I/O completes.

4. Memory Management and System Calls

Allocation

Reserves virtual address space for a process.

Protection

Prevents one process from accessing another process's private memory.

Virtual memory

Maps virtual pages to physical RAM and may use disk-backed paging.

Representative system calls / interfaces

- mmap() — maps files or anonymous memory into a process's virtual address space.
- munmap() — removes a memory mapping.
- brk() / sbrk() — adjusts the process data segment break on systems that support them; often avoided in modern application code.

Important distinction: malloc() is a C library function, not itself a system call; it may obtain memory using mmap() or brk().

6. I/O Operations and System Calls

Typical I/O path



Representative system calls

open() / close()

Open a file or device; release the file descriptor.

read() / write()

Transfer bytes between a file/device and a process buffer.

ioctl()

Performs device-specific control operations when supported.

select() / poll()

Waits for file descriptors to become ready for I/O.

The call may return immediately or block; drivers, interrupts, and buffering help coordinate slower devices.

7. Integrated Example, Benefits and Conclusion

Scenario: a text editor opens and reads a document

open()	The app requests access to the file; permissions are checked.
read()	The kernel requests data from storage through the I/O subsystem.
Waiting	If data is not ready, the process blocks and the scheduler runs another process.
Memory	Data is copied into a user-space buffer; virtual-memory protection isolates the app.
Resume	When I/O completes, the process becomes ready and can run again.